Tech Challenge — Fase 3
Guia de Deploy e Teste

Sequência completa para subir a infraestrutura na AWS, fazer o deploy da aplicação e validar o fluxo de autenticação via CPF — com os problemas reais encontrados e como evitá-los.

Grupo 15 — Pós-Graduação em Arquitetura de Software (FIAP/16SOAT)

1. Visão geral da arquitetura

O projeto está dividido em 4 repositórios GitHub, cada um com seu próprio CI/CD e branch `main` protegida (merge só via Pull Request):

Repositório	Responsável por
tech-challenge-infra-k8s	Terraform: VPC, cluster EKS, node group, ECR
tech-challenge-infra-db	Terraform: banco gerenciado (RDS PostgreSQL)
tech-challenge-auth-lambda	Lambda (Node.js) de autenticação via CPF + API Gateway
tech-challenge-app	API Spring Boot + manifests Kubernetes da aplicação

Ordem de dependência

Os repositórios não compartilham Terraform state entre si — cada um lê o que precisa do(s) outro(s) via **SSM Parameter Store** (parâmetros sob `/tech-challenge/*`). Por isso a ordem de aplicação importa:

```
1. tech-challenge-infra-k8s
   publica vpc_id, subnet_ids, eks_cluster_name
2. tech-challenge-infra-db
   le a rede do passo 1; publica rds_endpoint, rds_user, rds_password
3. tech-challenge-auth-lambda
   le a rede do 1 e o banco do 2; publica api_gateway_url, jwt_secret
4. tech-challenge-app
   deploy no cluster do passo 1, apontado pro RDS do passo 2
5. (de volta na Lambda) liga o proxy pra app do passo 4
```

■ **Dica:** Pra desligar tudo, a ordem é a **inversa**: auth-lambda → infra-db → infra-k8s. O tech-challenge-app não tem Terraform próprio — ele só aplica manifests Kubernetes dentro do cluster do infra-k8s, então some automaticamente quando o cluster é destruído.

2. Pré-requisitos

Ferramentas (instalar uma vez)

- **Terraform** ≥ 1.9 — terraform.io/downloads
- **AWS CLI v2** — `winget install Amazon.AWSCLI`
- **GitHub CLI** — `winget install GitHub.cli` (depois `gh auth login`)
- **kubectl** — `winget install Kubernetes.kubectl`
- **Docker Desktop** — precisa estar *aberto e rodando* antes de buildar imagens
- **Node.js** ≥ 20 — para empacotar a Lambda

Conta AWS Academy

O projeto roda inteiramente sob a role `LabRole` do AWS Academy Learner Lab — não cria nenhuma role ou policy IAM própria. Antes de começar, abra o Learner Lab (botão **Start Lab**, espere o círculo ficar verde).

■ **Atenção:** As credenciais do Learner Lab são **temporárias** e expiram em poucas horas (normalmente ~4h). Toda vez que expirarem, pegue um conjunto novo em **AWS Details** → **Show** e repita o passo 3 abaixo — isso **não** apaga nada que já foi criado, só impede novas chamadas até você renovar.

Clonando os repositórios

Sugestão: clone os 4 lado a lado numa mesma pasta.

```
git clone https://github.com/<sua-conta>/tech-challenge-infra-k8s.git
git clone https://github.com/<sua-conta>/tech-challenge-infra-db.git
git clone https://github.com/<sua-conta>/tech-challenge-auth-lambda.git
git clone https://github.com/<sua-conta>/tech-challenge-app.git
```

■ **Atenção:** No Windows, se algum `git clone`/`git add` falhar com "`Filename too long`", rode `git config --global core.longpaths true` e clone de novo (os caminhos de pacote Java do projeto passam do limite padrão de 260 caracteres do Windows).

Secrets do GitHub (para o CI funcionar)

Em cada um dos 4 repositórios, configure os secrets abaixo (Settings → Secrets and variables → Actions):

Secret	Onde	Valor
AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN	Todos os 4 repos	Credenciais do AWS Academy (expiram — atualizar periodicamente)
TF_VAR_DB_USER TF_VAR_DB_PASS	infra-db	Usuário/senha do RDS (defina os seus)
TF_VAR_JWT_SECRET	auth-lambda	Chave HMAC compartilhada com a app (mínimo 32 bytes)

```
gh secret set AWS_ACCESS_KEY_ID --repo "<conta>/<repo>" --body "<valor>"
# repita para AWS_SECRET_ACCESS_KEY, AWS_SESSION_TOKEN, etc.
```

3. Passo 0 — Credenciais da sessão

Antes de qualquer comando `terraform`/`aws`/`kubectl`, exporte as credenciais do Learner Lab **na janela de terminal que você vai usar** (variáveis de ambiente não valem entre janelas diferentes):

```
$env:AWS_ACCESS_KEY_ID = "ASIA..."
$env:AWS_SECRET_ACCESS_KEY = "..."
$env:AWS_SESSION_TOKEN = "..."
$env:AWS_DEFAULT_REGION = "us-east-1"

aws sts get-caller-identity   # confirma que validou
```

■ **Dica:** Se tiver vários passos ao longo do dia, só precisa repetir isso quando o `aws sts get-caller-identity` começar a dar `InvalidClientTokenId`.

4. Passo 1 — tech-challenge-infra-k8s (VPC + EKS + ECR)

Cria a rede e o cluster Kubernetes onde a aplicação vai rodar.

4.1 Bootstrap do bucket de state (só na primeira vez)

O bucket S3 do Terraform state **não é gerenciado pelo Terraform** — precisa ser criado manualmente via AWS CLI antes do primeiro `init`.

■ **Atenção:** Isso é proposital: o provider AWS sempre tenta ler a configuração de "object lock" do bucket como parte do ciclo de vida do recurso `aws_s3_bucket`, e o AWS Academy bloqueia essa chamada (`GetBucketObjectLockConfiguration`) via Service Control Policy, sem exceção possível. Se o Terraform tentar criar/gerenciar o bucket, o `apply` quebra nessa etapa — mesmo a criação em si funcionando. Por isso o bucket fica fora do Terraform.

Escolha um nome de bucket globalmente único (ex.: `tech-challenge-infra-k8s-<seu-usuario>`) e atualize `backend.tf` com ele. Depois:

```
aws s3api create-bucket \
  --bucket <nome-do-bucket> --region us-east-1

aws s3api put-bucket-versioning \
  --bucket <nome-do-bucket> \
  --versioning-configuration Status=Enabled

aws s3api put-bucket-encryption \
  --bucket <nome-do-bucket> \
  --server-side-encryption-configuration \
  '{"Rules":[{"ApplyServerSideEncryptionByDefault":
      {"SSEAlgorithm":"AES256"}}]}'

aws s3api put-public-access-block \
  --bucket <nome-do-bucket> \
  --public-access-block-configuration \
  BlockPublicAcls=true,IgnorePublicAcls=true,\
  BlockPublicPolicy=true,RestrictPublicBuckets=true
```

4.2 Aplicar

```
cd tech-challenge-infra-k8s
terraform init
terraform plan
terraform apply
```

Confirme com `yes`. A criação do cluster EKS demora **10–15 minutos**.

■ **Atenção:** Se o `apply` falhar no final com erros do tipo `"the server has asked for the client to provide credentials"` nos recursos `kubectl_manifest` (metrics-server, storage class): o token de autenticação do EKS expira em ~15 minutos, e a criação do cluster+node group consome esse tempo todo. **Solução: rode** `terraform apply` **de novo** — o cluster e o node group já existem, então a segunda passada é rápida e pega um token novo.

4.3 Verificar

```
terraform output configure_kubectl
# rode o comando que aparecer (aws eks update-kubeconfig ...)
kubectl get nodes   # deve mostrar os nós com status Ready
```

5. Passo 2 — tech-challenge-infra-db (RDS)

Precisa que o passo 1 já tenha sido aplicado (lê a VPC via SSM).

```
cd tech-challenge-infra-db
copy terraform.tfvars.example terraform.tfvars
# edite terraform.tfvars com db_user / db_pass reais

terraform init
terraform plan
terraform apply
```

Leva uns 5–10 minutos. Ao terminar, confira o endpoint:

```
terraform output rds_endpoint
```

6. Passo 3 — tech-challenge-auth-lambda (Lambda + API Gateway)

Precisa dos passos 1 e 2 já aplicados (lê VPC e RDS via SSM).

6.1 Empacotar a Lambda

```
cd tech-challenge-auth-lambda
npm ci --omit=dev
mkdir build
Compress-Archive -Path src, node_modules, package.json -DestinationPath build\lambda.zip
```

6.2 Configurar e aplicar

```
cd infra
copy terraform.tfvars.example terraform.tfvars
# edite terraform.tfvars: jwt_secret = "<chave com 32+ bytes>"
#                         app_backend_url = ""  (deixa vazio por enquanto)

terraform init
terraform plan
terraform apply
```

Rápido (Lambda + API Gateway sobem em segundos). Ao terminar:

```
terraform output auth_cpf_endpoint
```

6.3 Testar a emissão de token

O repositório `tech-challenge-app` já vem com um cliente de seed (migration V13) com CPF 52998224725, status ATIVO — dá pra testar assim que a app estiver no ar (passo 4):

```
curl.exe -X POST "<auth_cpf_endpoint>" `
  -H "Content-Type: application/json" `
  -d '{\"cpf\": \"52998224725\"}'
```

Antes do deploy da app (banco ainda sem tabelas), a resposta esperada é um erro de “tabela não existe” — confirma que Lambda → RDS já conectam, só falta o Flyway rodar.

7. Passo 4 — tech-challenge-app (deploy no EKS)

Precisa dos passos 1 e 2 já aplicados, e do `kubectl` apontando pro cluster (ver 4.3).

7.1 Apontar a URL do banco no ConfigMap

Edite `k8s/configmap.yaml`: troque `SPRING_DATASOURCE_URL` para apontar pro endpoint real do RDS (passo 2.3), com `?sslmode=require` no final:

```
SPRING_DATASOURCE_URL: >-
  jdbc:postgresql://<rds-endpoint>:5432/tech_challenge?sslmode=require
```

■ **Atenção:** O RDS exige conexão criptografada por padrão (`rds.force_ssl=1`). O driver JDBC do Postgres tenta SSL por padrão (`sslmode=prefer`), então normalmente funciona sem isso — mas deixe explícito pra evitar surpresa (a mesma coisa quebrou a Lambda, que usa `node-postgres`, cuja biblioteca **não** tenta SSL por padrão).

Troque também o tipo do Service pra `LoadBalancer` em `k8s/app-service.yaml` (já vem assim se você clonar depois da Fase 3 completa; confira se não está `ClusterIP`).

7.2 Publicar a imagem no ECR

Nos comandos abaixo, `<ecr>` = `<conta>.dkr.ecr.us-east-1.amazonaws.com` (a URL do ECR, vista em 4.2 via `terraform output ecr_repository_url`).

```
cd tech-challenge-app

aws ecr get-login-password --region us-east-1 | `
  docker login --username AWS --password-stdin <ecr>

docker build -t <ecr>/tech-challenge-backend:latest .

docker push <ecr>/tech-challenge-backend:latest
```

■ **Dica:** Precisa do Docker Desktop aberto e rodando. Se der erro de conexão com o daemon, abra o Docker Desktop e espere uns 30-60s.

7.3 Aplicar os manifests e ligar o segredo/imagem reais

`k8s/secret.yaml` no repositório é **só placeholder** de propósito (nunca commitamos segredo real) — o valor real é aplicado direto via `kubectl`, depois do `apply -k`:

```
kubectl apply -k k8s/

kubectl -n tech-challenge create secret generic tech-challenge-secrets `
  --from-literal=DB_USER=<usuario-do-rds> `
  --from-literal=DB_PASS=<senha-do-rds> `
  --from-literal=JWT_SECRET=<mesmo-jwt_secret-da-lambda> `
  --from-literal=MAIL_USERNAME=<opcional> `
  --from-literal=MAIL_PASSWORD=<opcional> `
  --dry-run=client -o yaml | kubectl apply -f -

kubectl -n tech-challenge set image `
  deployment/tech-challenge-app `
  app=<ecr>/tech-challenge-backend:latest

kubectl -n tech-challenge rollout status deployment/tech-challenge-app
```

■ **Atenção: MUITO IMPORTANTE:** toda vez que você rodar `kubectl apply -k k8s/` de novo (por exemplo, depois de um `git pull` com mudança no ConfigMap), ele **reaplica o secret.yaml placeholder e a imagem local** do repositório, **desfazendo** os dois comandos acima. Sempre repita o `create secret` e o `set image` logo em seguida.

Se o `JWT_SECRET` da app não for idêntico ao da Lambda, os tokens emitidos no fluxo de auth não vão validar.

7.4 Confirmar

```
kubectl -n tech-challenge get pods         # os 3 devem ficar 1/1 Running
kubectl -n tech-challenge get svc tech-challenge-app   # pega o EXTERNAL-IP (LoadBalancer)

curl.exe http://<external-ip-do-loadbalancer>/actuator/health
```

Resposta esperada: `{"status":"UP",...}`

8. Passo 5 — Ligar o proxy da Lambda na app

Com a app já exposta via LoadBalancer (passo 7.4), volta no repositório da Lambda e preenche a URL real:

```
cd tech-challenge-auth-lambda\infra
# edite terraform.tfvars:
#   app_backend_url = "http://<external-ip-do-loadbalancer>"

terraform plan   # deve mostrar 2 recursos novos (rota de proxy + integração)
terraform apply
```

9. Testando o fluxo completo

Com tudo no ar, o teste de ponta a ponta é:

```
Cliente (CPF)
  -> API Gateway POST /auth/cpf
  -> Lambda valida o CPF + consulta o RDS
  -> emite JWT
  -> API Gateway roteia requisicoes autenticadas (ANY /{proxy+})
  -> Lambda authorizer valida o JWT
  -> proxy ate a app no EKS
  -> app valida o JWT de novo (defesa em profundidade) + consulta o RDS
  -> retorna os dados
```

9.1 Emitir o token

```
curl.exe -X POST "<api_gateway_url>/auth/cpf" `
  -H "Content-Type: application/json" `
  -d '{\"cpf\": \"52998224725\"}'
```

Resposta esperada: `{"accessToken": "eyJ..."}`

9.2 Chamar uma rota protegida, através do Gateway

O cliente de seed (`52998224725`) tem o id `88888888-8888-4888-8888-888888888880` e já vem com uma ordem de serviço cadastrada:

```
$token = "<accessToken recebido acima>"

curl.exe "<api_gateway_url>/clientes/88888888-8888-4888-8888-888888888880/ordens" `
  -H "Authorization: Bearer $token"
```

Resposta esperada: JSON com a ordem de serviço da Ana Paula Ribeiro (veículo, mecânico, peças, serviços).

9.3 Outros testes úteis

- CPF com dígito verificador errado → `400`
- CPF válido mas inexistente no banco → `404`
- CPF de um cliente com `status = INATIVO` → `403`
- Rota protegida sem header `Authorization` → `401/403` do próprio Gateway
- Rota protegida com token adulterado/expirado → rejeitado pelo authorizer

■ **Dica:** Pra depurar a Lambda sem precisar chamar pela API, dá pra ver o log direto: `aws logs tail /aws/lambda/tech-challenge-auth-cpf --since 5m`

10. Problemas conhecidos (e como evitar)

Esses já estão corrigidos no código atual dos repositórios, mas documentamos aqui porque foram reais durante o desenvolvimento — úteis se algo parecido aparecer de novo.

EKS Access Policy Association falha com 404

Causa: aws_eks_access_entry e aws_eks_access_policy_association referenciavam a mesma ARN como string crua, sem depends_on — o Terraform podia tentar criar a associacao antes da entry existir.

Solução: Corrigido: depends_on explicito entre os dois recursos (access_entry.tf).

kubectl_manifest falha com 403 (forbidden)

Causa: Cascata do problema acima: sem a access policy association, a role voclabs nao tinha RBAC no cluster, entao qualquer kubectl apply (metrics-server, storage class) era rejeitado.

Solução: Corrigido: os kubectl_manifest tambem dependem da access policy association.

S3 bucket do state falha com AccessDenied (object lock)

Causa: O provider AWS sempre le GetBucketObjectLockConfiguration ao gerenciar um aws_s3_bucket. O AWS Academy bloqueia essa chamada via SCP, sem excecao.

Solução: Corrigido: o bucket nao e mais gerenciado pelo Terraform, e criado uma vez via AWS CLI (ver secao 4.1).

kubectl_manifest falha com “server has asked for credentials”

Causa: O token de autenticacao do EKS (data aws_eks_cluster_auth) e lido uma vez no inicio do apply e expira em ~15 minutos — a criacao do cluster+node group consome esse tempo.

Solução: Rode terraform apply uma segunda vez; cluster/node group ja existem, entao e rapido e pega um token novo.

Lambda retorna 500 “Erro interno ao validar CPF”

Causa: RDS exige SSL (rds.force_ssl=1); a biblioteca node-postgres nao tenta SSL a menos que configurado explicitamente.

Solução: Corrigido no codigo: ssl: { rejectUnauthorized: false } em src/db.js.

App em CrashLoopBackOff: “UnknownHostException: tech-challenge-postgres”

Causa: O ConfigMap aplicado no cluster estava desatualizado (git pull nao tinha sido rodado antes do kubectl apply).

Solução: Sempre conferir com git log / git pull antes de reaplicar manifests.

App em CrashLoopBackOff: “password authentication failed”

Causa: kubectl apply -k k8s/ reaplicou o secret.yaml placeholder do repo, sobrescrevendo o Secret real criado manualmente.

Solução: Sempre repetir kubectl create secret (+ set image) logo depois de qualquer kubectl apply -k k8s/ (ver secao 7.3).

git add falha com “Filename too long” no Windows

Causa: Caminhos de pacote Java (com.fiap.tech_challenge_backend....) somados a diretorios profundos passam do limite de 260 caracteres do Windows.

Solução: git config --global core.longpaths true antes de clonar/commitar.

11. Desligando tudo (evitar custo)

O cluster EKS, os nodes e o RDS geram cobrança contínua enquanto existirem. Duas opções, dependendo de quanto tempo vai ficar parado:

Opção A — Pausa curta (algumas horas): zerar os nodes do EKS

O control plane do EKS continua cobrando uns centavos por hora (não dá pra pausar ele), mas os nodes (o custo maior) podem ser zerados:

```
aws eks update-nodegroup-config --cluster-name eks-tech-challenge \
  --nodegroup-name nodeg-tech-challenge \
  --scaling-config minSize=0,maxSize=3,desiredSize=0

# para religar depois:
aws eks update-nodegroup-config --cluster-name eks-tech-challenge \
  --nodegroup-name nodeg-tech-challenge \
  --scaling-config minSize=2,maxSize=3,desiredSize=2
```

Opção B — Pausa longa (dias): destruir tudo

Zero custo, mas precisa recriar do zero depois. **Ordem inversa** da subida — lambda e banco primeiro, cluster por último (eles dependem da rede dele via SSM):

```
cd tech-challenge-auth-lambda\infra  &&  terraform destroy
cd tech-challenge-infra-db           &&  terraform destroy
cd tech-challenge-infra-k8s          &&  terraform destroy
```

■ **Dica:** O tech-challenge-app não precisa de destroy separado — ele some junto quando o cluster do infra-k8s é destruído.